

claude-windows / a9fd4426-f234-4e4a-bfb4-1f2c8255c661

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\a9fd4426-f234-4e4a-bfb4-1f2c8255c661\subagents\workflows\wf_076beba8-59c\agent-a4230a4dd88b5c843.jsonl`  
- SHA-256: `39747e67b908350ffbe158a5cc7cde5b966f7123644e700f98938b6722efa58c`  
- Source modified: `2026-05-31T04:19:36+00:00`  
- Imported at: `2026-07-05T16:48:20+00:00`  
- project: `wf_076beba8-59c`  
- session_id: `a9fd4426-f234-4e4a-bfb4-1f2c8255c661`
```

Transcript

1. user (2026-05-31T04:18:54.601Z)

Source Extractor

Research question: "Identify the best LOCAL (self-hosted, single NVIDIA CUDA GPU) vision / document-AI / OCR models ? and, as a clearly-flagged approval-gated fallback only, paid cloud API services ? for parsing BANK STATEMENT PDFs into structured transaction tables (date, narration/description, withdrawal/debit, deposit/credit, running balance), for a LOCAL-FIRST personal-finance tool ("muneem").

Target documents: Indian bank statements (HDFC, Axis, ICICI, AU Small Finance) ? dense, multi-column, often SEMI-RULED or borderless tables, sometimes multiple accounts per statement; both born-digital (text layer present) and the harder cases where pdfplumber's extract_tables/coordinate clustering is unreliable (e.g. Axis, whose column layout varies between statements).

HARD CONSTRAINTS (a model that violates these is unusable for us):

1. MUST run 100% LOCALLY for statement contents (highly sensitive). NO cloud APIs by default. Paid/cloud is acceptable ONLY as an explicitly-approved fallback ? still report the best cloud options but label them clearly as "cloud/sensitive ? approval-gated, not default".
2. MUST be usable from Python (transformers / vLLM / Ollama / ONNX Runtime / native lib).
3. License MUST be permissive & commercially safe for BOTH the CODE and the MODEL WEIGHTS: MIT, BSD, Apache-2.0, ISC, HPND. AVOID and explicitly FLAG anything GPL/LGPL/AGPL, CC-BY-NC / non-commercial, research-only, or custom "community"/acceptable-use licenses with restrictions. This is critical ? many document-AI models and their training datasets are non-commercial (e.g. LayoutLMv3 weights are CC-BY-NC; Surya and Marker use GPL-or-commercial dual licensing; several VLM weights ship custom restrictive licenses). Verify the WEIGHTS license, not just the repo license.
4. Hardware: a single consumer/prosumer NVIDIA GPU. Give VRAM tiers (~8-12 GB consumer, ~16-24 GB) and which models/quantizations fit each.

Evaluate and compare these LOCAL approaches ? for each give: code license, MODEL-WEIGHTS license (state explicitly), latest release / maintenance status as of 2025-2026, VRAM need + quantization options, Python integration path, and reliability specifically on DENSE FINANCIAL TABLES / multi-column statements:

- Open vision-language models (VLMs) for document/table understanding: Qwen2-VL and Qwen2.5-VL (note per-size license ? which sizes are Apache-2.0 vs restricted), MiniCPM-V 2.6, InternVL2 / 2.5, Microsoft Phi-3.5-vision, Mistral Pixtral, Meta Llama-3.2-Vision (flag the Llama Community License restrictions), Moondream2, GOT-OCR2.0, olmOCR and other OCR-specialized VLMs.
- OCR-specialist / document-AI pipelines: PaddleOCR + PP-Structure (table recognition), docTR, EasyOCR, Tesseract (already used), Microsoft Table Transformer / TATR (check code AND PubTables-1M dataset/weights license), Donut, Surya (FLAG license), Marker (FLAG license), MinerU / PDF-Extract-Kit, RapidOCR.
- Table-structure recognition specifically (cell/grid reconstruction for borderless tables): TATR, PP-Structure, UniTable, etc.

Also assess: for SEMI-RULED / borderless bank tables, is a prompt-driven VLM (page image -> JSON

transactions) more reliable than a classic OCR + table-structure pipeline? What are the accuracy/hallucination risks of VLMs on financial NUMBERS (transposed digits, dropped/duplicated rows), and how should output be VERIFIED? (muneem already machine-checks parses by balance reconciliation ? opening + sum(deposits) - sum(withdrawals) == closing, plus per-row running-balance walks and cross-checks against the bank's printed summary ? so any model's output can be automatically validated, which should inform the recommendation.)

Finally, briefly cover the best PAID/CLOUD fallbacks (clearly flagged approval-gated for sensitive data): AWS Textract (AnalyzeExpense / Tables), Google Document AI, Azure AI Document Intelligence, Reducto, LlamaParse, Anthropic/OpenAI vision ? note any self-hosting option and data-handling terms.

Deliver a RANKED shortlist of the most promising LOCAL options for muneem (NVIDIA GPU, Python, commercial-license-safe), each with license (code + weights), VRAM tier, maturity, strengths/weaknesses for dense bank-statement tables, and a concrete recommendation: which local model(s) to prototype for muneem's statement parsing and how they'd combine with the existing deterministic parsers + balance-reconciliation verification (e.g. VLM as a fallback/cross-check when deterministic parsing fails to reconcile). Cite sources and call out any license uncertainty explicitly."

Fetch and extract key claims from this source:

```
**URL:** https://logic.inc/resources/llm-table-extraction
**Title:** LLM Table Extraction: Own the Infrastructure or Offload It (Logic.inc)
**Found via:** Borderless-table accuracy & financial-number hallucination/verification
(contrarian/skeptical) search
```

Task

1. Use WebFetch to retrieve the page content.
2. Assess source quality: primary research/institution? secondary reporting? blog/opinion? forum? unreliable?
3. Extract 2-5 FALSIFIABLE claims that bear on the research question. Each claim must:
 - be a concrete, checkable statement (not vague generalities)
 - include a direct quote from the source as support
 - be rated central/supporting/tangential to the research question
4. Note publish date if available.

If the fetch fails or the page is irrelevant/paywalled, return claims: [] and sourceQuality: "unreliable".

Structured output only.

2. assistant (2026-05-31T04:18:58.339Z)

I'll fetch the source page and extract the key claims. Let me load the WebFetch tool first.

3. assistant (2026-05-31T04:19:00.567Z)

I have the WebFetch tool loaded. Now let me fetch the source page.

4. user (2026-05-31T04:19:14.769Z)

Key Claims Extraction: LLM Table Extraction Article

****Source:**** Logic blog post by Marcus Fields, published March 13, 2026 (updated March 15, 2026). This is a vendor blog promoting Logic's commercial platform.

(1) Accuracy & Reliability on Complex Tables

****Claim:**** LLMs underperform on structurally complex documents.

> "Merged cells, nested headers, and tables spanning multiple pages cause hallucinations even in

the best available models."

The article asserts frontier models "substantially underperform traditional optical character recognition (OCR) approaches on accuracy, speed, and cost" for complex production documents like construction drawings and financial statements.

(2) Hallucination & Consistency Risks

****Claim:**** Multiple failure modes identified for batch processing.

> "In multi-page document sets, pages randomly miss rows, field ordering shifts between runs, and the same document processed twice can produce different outputs."

The article also notes: > "At batch scale, per-document error rates that look acceptable in isolation compound into a significant daily backlog of records requiring investigation."

(3) Validation & Verification Methods

****Claim:**** Multi-stage validation required for production reliability.

> "Production pipelines require at least three validation stages to reach reliable results: missing attribute checks, grounding verification (confirming extracted values exist in the source), and rule compliance checks."

The article recommends testing with "labeled tables with cell-level ground truth" and "regression detection across three independent drift sources: prompt changes, model updates, and document format changes."

(4) Own vs. Offload Decision Framework

****Claim:**** Ownership justifiable only in specific scenarios.

> "If extraction is what you sell, own it. If extraction enables what you sell, offload the infrastructure and keep engineers on the work that matters."

Offloading recommended when "extraction quality is not your competitive differentiator" or when "standard document formats are" handled reliably by third parties.

(5) Specific Models & Benchmarks

****No specific accuracy benchmarks or model comparisons provided.**** The article mentions GPT, Claude, Gemini, and Perplexity as routed providers but provides no quantitative performance data, benchmarks, or head-to-head comparisons between models.

(6) Borderless/Semi-Ruled Table Handling

****Not addressed**** in this article. No discussion of table border detection or handling semi-structured tables.

5. assistant (2026-05-31T04:19:19.756Z)

I have the extracted content. Let me assess and structure the findings.

The source is a vendor blog from Logic.inc (author Marcus Fields, March 2026) promoting their commercial extraction platform. Despite the vendor bias, it makes several falsifiable, on-point claims about LLM/VLM reliability on dense financial tables, hallucination modes, and verification strategy ? all central to the research question. The "own vs offload" framing and absence of benchmarks reflect its commercial-blog nature.

6. user (2026-05-31T04:19:26.318Z)

Structured output provided successfully

7. assistant (2026-05-31T04:19:36.762Z)

Source processed. Summary of what I found at <https://logic.inc/resources/llm-table-extraction>:

****Source quality: blog (vendor)**** ? A Logic.inc company blog post by Marcus Fields, published 2026-03-13 (updated 2026-03-15), promoting their commercial extraction platform. Treat its "own vs offload" framing as commercially motivated, but its technical claims about LLM failure modes are concrete and align with the contrarian/skeptical angle the search targeted.

****5 falsifiable claims extracted**** (3 central, 2 supporting), all bearing on the VLM-reliability and verification parts of the research question:

- ****Central ? hallucination on complex tables:**** Merged cells, nested headers, and multi-page tables cause hallucinations "even in the best available models," and frontier LLMs "substantially underperform traditional OCR" on accuracy/speed/cost for complex docs including financial statements.
- ****Central ? non-determinism / dropped rows:**** "pages randomly miss rows, field ordering shifts between runs, and the same document processed twice can produce different outputs." This directly substantiates muneem's concern about dropped/duplicated rows from VLMs.
- ****Central ? three-stage validation:**** missing-attribute checks, grounding verification (values must exist in source), and rule-compliance checks. This maps onto muneem's balance-reconciliation verification approach.
- ****Supporting ? batch-scale error compounding:**** per-doc error rates acceptable in isolation compound into significant manual-review backlog.
- ****Supporting ? drift sources:**** cell-level ground-truth labels plus regression detection across prompt/model/format drift.

****Caveats / limits:**** The article provides ****no quantitative benchmarks or model-vs-model comparisons**** (it only name-drops GPT, Claude, Gemini, Perplexity as routed providers), and ****does not address borderless/semi-ruled table handling specifically**** ? so it supports the ***skeptical case against VLMs and the case for automated verification***, but offers no data to rank specific local models. No code/weights licensing content here.